

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет о выполнении практической работы №4
по дисциплине «Функциональное программирование»
Вариант №7.

Студент,
группы 5130201/30101

_____ Мартыненко А. П.

Преподаватель

_____ Моторин Д.Е.

«_____» _____ 2025г.

Санкт-Петербург, 2025

Содержание

Введение	3
1 Постановка задачи	4
2 Разработка модели генеалогического дерева	5
2.1 Типы данных	5
2.2 Работа с файлом	5
2.3 Поиск в глубину	6
3 Проверка корректности	8
Выводы	10
Список использованных источников	11
Приложение 1 «Исходный код программы»	12

Введение

Генеалогическое дерево - это схема, которая представляет родственные связи потомков и предков.

Генеалогическое дерево часто представляется в виде графа, в котором у «корней» указываются предки, а каждый «лист» - потомок по конкретной генеалогической линии.

В каждый элемент может входить:

1. Фамилия (обязательно), имя, отчество (при наличии).
2. Годы жизни.
3. Фотография человека.
4. Место рождения, место смерти

Пример построения генеалогического дерева представлен на рис. 1.

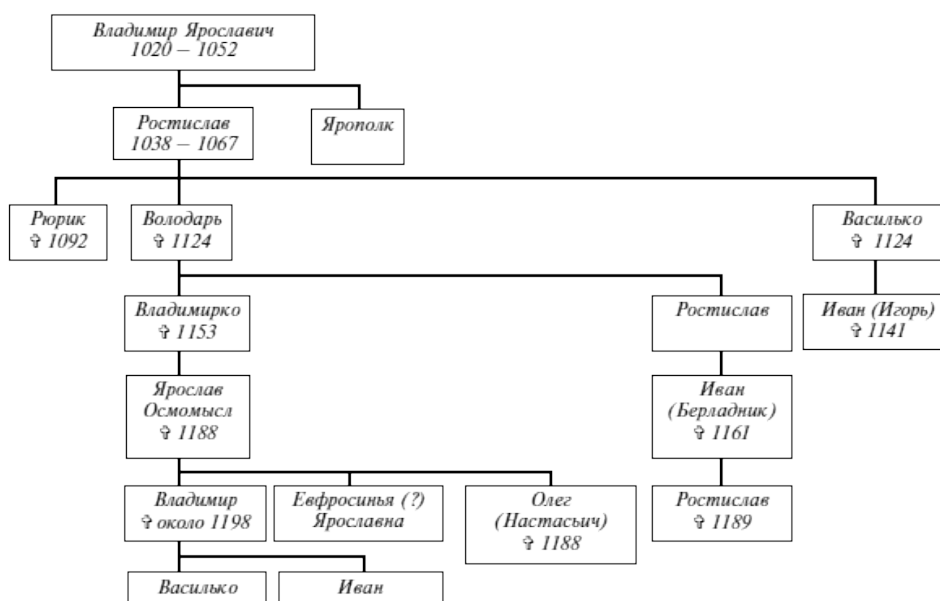


Рис. 1. Генеалогическое дерево

Генеалогические деревья используются для отслеживания происхождения, нахождения предков и визуализации семейной истории.

1 Постановка задачи

Цель: реализация программы поиска по генеалогическому дереву родственников.

Задачи:

1. Определить тип представления родственников Relative и ParentsTable.
2. Написать функцию readFile.
3. Написать функцию поиска в глубину findDepth.

2 Разработка модели генеалогического дерева

2.1 Типы данных

Для работы определены типы данных для представления генеалогического дерева.

Листинг 1. Типы данных

```
1 type Relative = (String, String)
2 type ParentsTable = [Relative]
```

Тип `Relative` представляет родственное отношение как кортеж двух строк (родитель-потомок), то есть два узла дерева, соединенные ребром. Тип `ParentsTable` - список таких отношений, который представляет генеалогическое дерево в целом.

2.2 Работа с файлом

Функция `readMyFile` предназначена для чтения данных из файла. Для чтения строк из файла используется явное управление файловым дескриптором. Функция с помощью `openFile` открывает файл в режиме чтения и возвращает файловый дескриптор `handle`. С помощью `hGetContents` она читает все содержимое файла. Так как `hGetContents` читает данные постепенно, необходимо использовать строгую оценку длины (принудительно заставить систему пройти по всему содержимому файла). Затем функция парсит данные и явно закрывает файловый дескриптор `hClose`.

Листинг 2. Функция `readMyFile`

```
1 readMyFile :: FilePath -> IO ParentsTable
2 readMyFile filename =
3   openFile filename ReadMode >>= \handle ->
4   hGetContents handle >>= \content ->
5   let parsed = parse content
6   !_ = length content
7   in hClose handle >> return parsed
```

Функция `parse` преобразует содержимое файла в структуру данных `ParentsTable`.

Листинг 3. Функция `parse`

```
1 parse :: String -> ParentsTable
2 parse content =
3   map parseLine (filter (not . null) (lines content))
4   where
5     parseLine line =
```

```

6 case break (== '-') line of
7 (parent, '-':rest) ->
8 let child = dropWhile (== ' ') rest
9 in (trim parent, trim child)
10 _ -> (" ", "")
11 trim = unwords . words

```

В текстовом файле каждая строка имеет вид «ФИ предка» - «ФИ последователя». Функция разбивает содержимое на строки по символам перевода строки и каждую непустую строку разбивает на два элемента по «-».

2.3 Поиск в глубину

Функция `findChildren` возвращает всех детей (непосредственных потомков) заданного человека в генеалогическом дереве.

Листинг 4. Функция `findChildren`

```

1 findChildren :: String -> ParentsTable -> [String]
2 findChildren person relation =
3 [child | (parent, child) <- relation, parent == person]

```

Функция `findDepth` реализует поиск родственников в глубину. Она возвращает список всех найденных потомков до указанной глубины поиска. Значения из файла записываются во внешнее окружение монады `Reader` через `ask`, передавая таблицу `ParentsTable` через все рекурсивное дерево без явной передачи аргумента на каждом шаге.

Листинг 5. Функция `findDepth`

```

1 findDepth :: String -> Int -> Reader ParentsTable [String]
2 findDepth start maxDepth =
3 search start 0 []
4 where
5 search :: String -> Int -> [String] -> Reader ParentsTable [String]
6 search curPerson curDepth visited
7 | curDepth >= maxDepth = return []
8 | curPerson `elem` visited = return []
9 | otherwise =
10 ask >>= \relat ->
11 let children = findChildren curPerson relat
12 newVisited = curPerson : visited
13 in mapM (\child -> search child (curDepth+1) newVisited) children >>= \
14     next ->
15 return (nub (children ++ concat next))

```

Вложенная функция `search` реализует рекурсивный поиск в глубину. При достижении максимальной глубины или обнаружении уже обработанного человека поиск прекращается. Иначе функция получает доступ

к таблице отношений, находит всех детей текущего человека, рекурсивно продолжает поиск для каждого ребенка с увеличенной глубиной и объединяет итоговые результаты.

3 Проверка корректности

Для тестирования корректности функции поиска в глубину создан файл, содержащий информацию о генеалогическом дереве. Его графическое представление показано на рисунке 2.

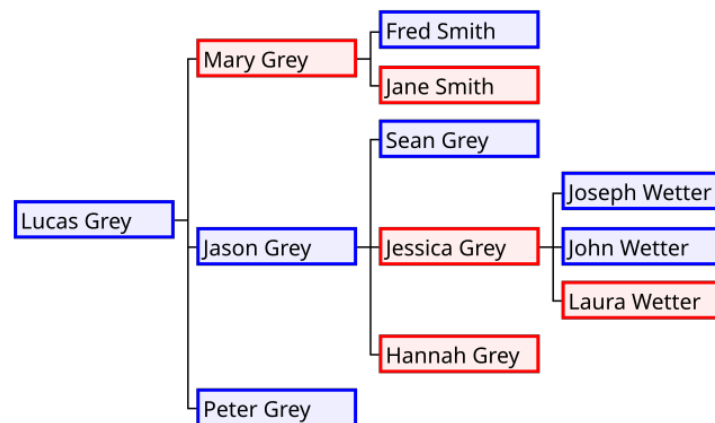


Рис. 2. Генеалогическое дерево

На рисунках 3 - 5 приведены результаты работы программы в случаях поиска от предка глубиной 1 и 2, а также поиск потомков для человека, у которого нет потомков.

```
Enter name for search: Lucas Grey
Enter search depth: 1

Results for Lucas Grey (depth 1):
Mary Grey
Jason Grey
Peter Grey
```

Рис. 3. Поиск глубиной 1


```
Enter name for search: Lucas Grey
Enter search depth: 2

Results for Lucas Grey (depth 2):
Mary Grey
Jason Grey
Peter Grey
Fred Smith
Jane Smith
Sean Grey
Jessica Grey
Hannah Grey
```

Рис. 4. Поиск глубиной 2

```
Enter name for search: John Wetter
Enter search depth: 1

Results for John Wetter (depth 1):
No relatives found
```

Рис. 5. Поиск с отсутствием потомков

Тестирование подтвердило, что функция работает корректно, обрабатывая ситуации наличия и отсутствия потомков.

Выводы

В ходе работы релизована программа поиска в глубину по генеалогическому дереву родственников. Определен тип представления родственников `Relative` и `parentsTable`, написаны функции работы с файлом `readFile` и функция поиска в глубину `findDepth`. Тестирование программы показало, что все функции работают корректно и обрабатывают частные случаи.

Список литературы

- [1] Курт У. «Программируй на Haskell». М.: Издательство, 2019. 320 стр.
- [2] Липовача М. «Изучай Haskell во имя добра». М.: ДМК Пресс, 2012. 490 стр.

Приложение 1 «Исходный код программы»

Листинг 6. Код работы

```
1 {-# LANGUAGE BangPatterns #-}
2 module Tree where
3 import Control.Monad.Reader (Reader, runReader, ask)
4 import Data.List
5 import System.IO
6
7 type Relative = (String, String)
8 type ParentsTable = [Relative]
9 findChildren :: String -> ParentsTable -> [String]
10 findChildren person relation =
11   [child | (parent, child) <- relation, parent == person]
12
13 findDepth :: String -> Int -> Reader ParentsTable [String]
14 findDepth start maxDepth =
15   search start 0 []
16 where
17   search :: String -> Int -> [String] -> Reader ParentsTable [String]
18   search curPerson curDepth visited
19     | curDepth >= maxDepth = return []
20     | curPerson `elem` visited = return []
21     | otherwise =
22       ask >>= \relat ->
23       let children = findChildren curPerson relat
24       newVisited = curPerson : visited
25       in mapM (\child -> search child (curDepth+1) newVisited) children >>=
26         \next ->
27         return (nub (children ++ concat next))
28
29 readMyFile :: FilePath -> IO ParentsTable
30 readMyFile filename =
31   openFile filename ReadMode >>= \handle ->
32   hGetContents handle >>= \content ->
33   let parsed = parse content
34   ! _ = length content
35   in hClose handle >> return parsed
36
37 parse :: String -> ParentsTable
38 parse content =
39   map parseLine (filter (not . null) (lines content))
40 where
41   parseLine line =
42     case break (== '-') line of
43       (parent, '-':rest) ->
44         let child = dropWhile (== ' ') rest
45         in (trim parent, trim child)
46       _ -> ("", "")
47   trim = unwords . words
48
49 findRelatives :: String -> Int -> FilePath -> IO [String]
50 findRelatives person depth filename =
51   readMyFile filename >>= \relationships ->
52   return (runReader (findDepth person depth) relationships)
```